

Basic (your first graph) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangGraph), **Advanced** (how state merging actually works once more than one field is involved). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A graph is just: a shape for the data (the state), some functions that each do one small piece of work (nodes), and connections saying what runs after what (edges).

For your first graph: one field in the state, two nodes that each change that field a little, one edge connecting node 1 to node 2. Then you compile the graph (turns it into something runnable) and invoke it (actually runs it, start to finish) with a starting value.

Things to use:

- `TypedDict` — defines the shape of your state.
- `StateGraph(YourStateType)` — starts building a graph around that shape.
- `graph.add_node("name", function)` — registers one node.
- `graph.add_edge("from", "to")` — connects two nodes.
- `graph.compile()` then `.invoke({...})` — makes it runnable, then runs it once.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Every LangGraph graph has the same 3 ingredients: a **state shape**, **nodes** (plain functions, state in, a small update out), and **edges** (what runs next). A `StateGraph` needs a real entry point and exit point too — `START` and `END`, both imported from `langgraph.graph`, are special markers, not nodes you write yourself.

The exact pieces:

- `from typing import TypedDict` and `class GraphState(TypedDict): message: str` — your state's shape, as a real type.
- `from langgraph.graph import StateGraph, START, END`
- `builder = StateGraph(GraphState)` — starts the builder, tied to your state shape.
- `def node_one(state: GraphState) -> dict: return {"message": ...}` — a node takes the whole state, returns only the keys it's changing (a **partial update**), not the whole state back.
- `builder.add_node("node_one", node_one)` and the same for `node_two`.
- `builder.add_edge(START, "node_one")`, `builder.add_edge("node_one", "node_two")`, `builder.add_edge("node_two", END)` — explicit start and end, not implied by node order.
- `graph = builder.compile()` then `graph.invoke({"message": "..."})`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what "a node returns a small update" actually means the moment your state has more than one field, or a field that's a list you want to *grow*, not replace. By default, LangGraph merges a node's returned dict into state one key at a time — for a plain field like `message: str`, that means the new value simply overwrites the old one. But what if two different nodes both need to append to the *same* running log, and you don't want node 2's write to erase node 1's?

The real design question isn't just "how do nodes update state" — it's "for each field, should a new node's update *replace* what's there, or *combine* with it?"

The extra piece that answers that question:

- **Reducers**, declared right in the state's type: `from typing import Annotated; import operator; log: Annotated[list[str], operator.add]`. Now, instead of overwriting `state["log"]` with whatever a node returns, LangGraph calls `operator.add(old_log, new_log)` — for lists, that's concatenation — so each node's returned `{"log": ["node_one ran"]}` *appends*, instead of replacing the whole history. Without a reducer, the default behavior is a plain overwrite, which is exactly right for a field like `message` but silently wrong for a field you meant to accumulate.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both use a single plain-`str` field, where "overwrite" is exactly the right, unsurprising behavior. Advanced asks what happens the moment a second field needs the opposite behavior — accumulate instead of overwrite — and shows the one annotation (`Annotated[... , operator.add]`) that changes a field's merge behavior from "replace" to "combine," which becomes essential the moment your state needs to track a running history (a message list, a list of sources found) instead of a single current value.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a state shape with one field: message (text)

make node_one: takes state, returns {"message": "Hello"}
make node_two: takes state, returns {"message": state's message + ", World!"}

build the graph:
    start it with the state shape
    add node_one, add node_two
    connect: start -> node_one -> node_two -> end

run it:
    compile the graph
    invoke it with a starting message
    print what comes back -- trace by hand what "message" should be after each no
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# first_graph_practice.py
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    message: str

def node_one(state):
    return {"message": "Hello"}

def node_two(state):
    return {"message": state["message"] + ", World!"}

builder = StateGraph(GraphState)
builder.add_node("node_one", node_one)
builder.add_node("node_two", node_two)
```

Expected output if you run just this: nothing — add the 3 `add_edge(...)` calls, `compile()`, and an `invoke({"message": "start"})` call with a `print(...)` to see `{'message': 'Hello, World!'}`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

define:
    class GraphState(TypedDict):
        message: str

define:
    def node_one(state: GraphState) -> dict:
        return {"message": "Hello"}

define:
    def node_two(state: GraphState) -> dict:
        return {"message": state["message"] + ", World!"}

build:
    builder = StateGraph(GraphState)
    builder.add_node("node_one", node_one)
    builder.add_node("node_two", node_two)
    builder.add_edge(START, "node_one")
    builder.add_edge("node_one", "node_two")
    builder.add_edge("node_two", END)

run:
    graph = builder.compile()
    result = graph.invoke({"message": "start"})
    print(result)
```

Trace it by hand before running: what should `state["message"]` be right after `node_one` runs, and again right after `node_two` ? Then run it and compare against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

add a second field, `log`, that should accumulate instead of overwrite:

```
class GraphState(TypedDict):
    message: str
    log: Annotated[list[str], operator.add]
```

each node now also returns `{"log": ["node_one ran"]}` or `{"log": ["node_two ran"]}`

run it and check: does `state["log"]` end up with BOTH entries, in order, or does `node_two`'s write erase `node_one`'s?

```
# first_graph_practice.py
from typing import TypedDict, Annotated
import operator
from langgraph.graph import StateGraph, START, END

class GraphState(TypedDict):
    message: str
    log: Annotated[list[str], operator.add]

def node_one(state: GraphState) -> dict:
    return {"message": "Hello", "log": ["node_one ran"]}

def node_two(state: GraphState) -> dict:
    return {"message": state["message"] + ", World!", "log": ["node_two ran"]}
```

Wire this into a full graph yourself (same edges as before), invoke it with `{"message": "start", "log": []}`, and confirm `result["log"]` has both entries in order — then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate's single `message` field only ever needs the default overwrite behavior. Advanced adds a second field, `log`, that needs the opposite — declared with `Annotated[list[str], operator.add]` so each node's write combines with what's already there instead of replacing it, proving both merge behaviors side by side in the same state shape.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)